CPU Scheduler Simulation – Project Report

Introduction

This project implements a CPU scheduling simulator in Java. All processes arrive at time 0 and are executed using three scheduling algorithms:

1. Shortest Job First (Non-Preemptive),
2. Round Robin with a quantum of 7 milliseconds,
3. Priority Scheduling with starvation detection and aging.

Each process is represented using a Process Control Block (PCB) that stores its burst time, priority, memory size, and timing values such as start time, waiting time, and turnaround time.
Multithreading is used to simulate real operating system behavior where job loading and scheduling happen in parallel.

Software and Hardware Tools Used

The project was implemented using the Java programming language (JDK).
VS Code or IntelliJ was used as the development environment.
Execution and testing were done on a Windows PC or laptop with a multi-core processor and at least 8GB of RAM.

System Calls Used

To simulate OS behavior, the program included several system-like operations such as admitting a process to the ready queue, dispatching a process to the CPU, preempting a running process in Round Robin, terminating the process when finished, and applying aging to increase process priority and avoid starvation.
These calls help track the progress of each process and store useful timing information for evaluation.

Multithreading Justification

The simulator uses separate threads that work together at the same time.
One thread reads the job.txt file and inserts processes into the Job Queue.
Another thread checks memory availability and moves jobs into the Ready Queue whenever possible.
Meanwhile, the main thread executes the scheduler and prints the results.

This parallelism improves performance because the scheduler does not wait for the input file to finish loading. It also makes the system behave more like an actual operating system.

Strengths and Weaknesses of the Program

Strengths:
The system supports three scheduling techniques correctly. It prevents starvation by using aging. It uses multiple threads which adds realism. It prints a Gantt chart and calculates average waiting time and average turnaround time. It also saves results to a CSV file for further evaluation.

Weaknesses:
Some scheduling algorithms are non-preemptive. Memory management is simplified instead of using paging or segmentation. Context switching overhead is not simulated.

Output and Performance Analysis

For each scheduling method, the program displays a Gantt chart showing the order and timing of execution.
It also prints the average waiting time and average turnaround time.
In priority scheduling, the system also detects if a process has suffered starvation.

Using the Gantt chart provides a clear visual comparison between the algorithms in terms of how efficiently the CPU is being used.

Conclusion

This project successfully demonstrates CPU scheduling concepts through a Java implementation. The use of multithreading and PCB data structures makes the simulation represent real scheduling behavior. Aging is used to avoid starvation, and performance metrics are measured accurately using waiting time and turnaround time.
Overall, the simulator provides a powerful educational tool for understanding CPU schedulers in operating systems.

Prepared by:

1- SULTAN SAEED ALQAHTANI \ 444100168

2- ABDULAZIZ MOHAMMED ALDURAIBI \ 444102549

3- YAZEED SAEED ALQARNI \ 444101099

Course: CSC227 - Operating Systems

Project Title: CPU Scheduler Simulation

Date: November 2025